

Part 1: ICMP and Ping

1.) What is the IP address of your host? What is the IP address of the destination host?

The IP of my host is 192.168.1.171 and the IP of the destination host is 143.89.209.9 as seen in the screenshot below.

No.	Time	Source	Destination	Protocol	Length	Info
5098	5.013843	179.108.19.22	192.168.1.171	ICMP	174	Destination unreachable (Host unreachable)
7444	8.154835	217.142.22.132	192.168.1.171	ICMP	174	Destination unreachable (Port unreachable)
→ 7860	9.184969	192.168.1.171	143.89.209.9	ICMP	74	Echo (ping) request id=0x0001, seq=11/2816, ttl=128 (reply in 7956)

2.) Why is that an ICMP packet does not have source and destination port numbers?

The ICMP packet doesn't have a port number because it doesn't communicate to any application in the destination. Its main purpose is to check the speed of the connection, so there is no need for it to give information to an application/process.

3.) Examine one of the ping request packets sent by your host.

a.) What are the ICMP type and code numbers?

The ICMP type is 8 and the code number is 0 as seen in the screenshot below.

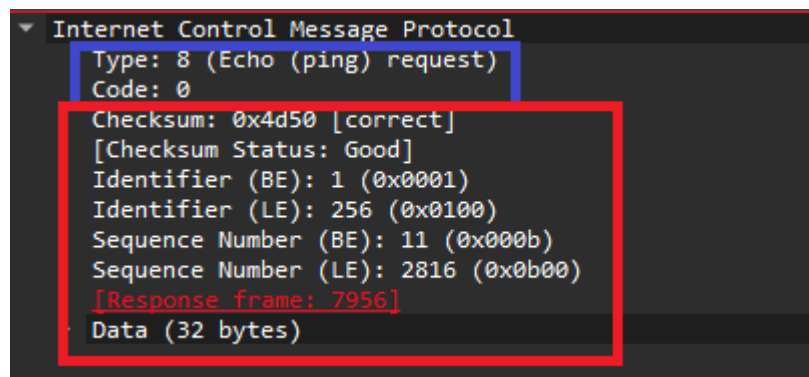


Figure 1: The type and code numbers (blue) and the subsequent fields (red) for the ping request packet

b.) What other fields does the ICMP packet have?

As seen in figure 1, the other fields are Checksum, Checksum status, Identifier (BE), Identifier (LE), Sequence Number (BE), and Sequence Number (LE).

c.) How many bytes are the checksum, sequence number and identifier fields?

The checksum, sequence number, and identifier fields are 2 bytes each as seen in the screenshot above.

4.) **Examine the corresponding ping reply packet.**

a.) **What are the ICMP type and code numbers?**

The type and code numbers are both 0 for the reply packet as seen in figure 2.

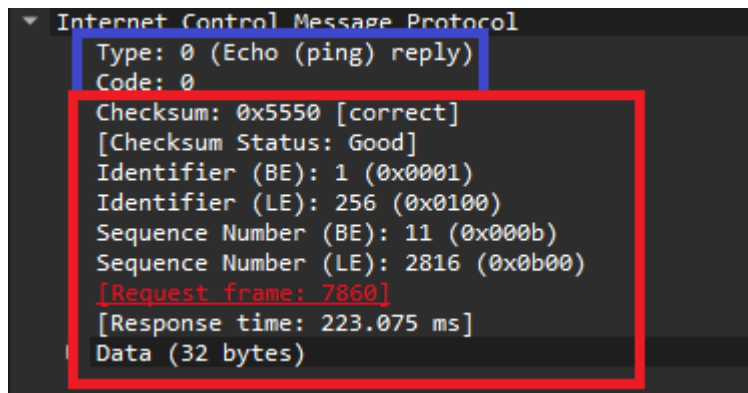


Figure 2: The type and code numbers (blue) and subsequent fields (red) for the reply packet.

b.) **What other fields does this ICMP packet have?**

The ICMP packet has Checksum, Checksum status, Identifier (BE), Identifier (LE), Sequence Number (BE), Sequence Number (LE), and Response Time. This can be seen in figure 2.

c.) **How many bytes are the checksum, sequence number and identifier fields?**

The checksum, sequence number, and identifier fields are all 2 bytes as seen in figure 2.

Part 2: ICMP and Traceroute (tracert)

1) What is the IP address of your host? What is the IP address of the target destination host?

Host IP address: 10.0.0.216

Destination IP address: 128.93.162.83

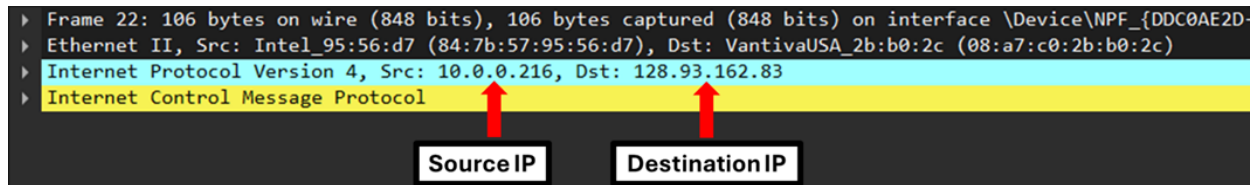


Figure 3: Source and Destination IP address

2) If ICMP sent UDP packets instead (as in Unix/Linux), would the IP protocol number still be 01 for the probe packets? If not, what would it be?

No, the protocol number will not be 1 if UDP packets are sent instead. ICMP uses protocol number 1 (0x01 as seen in the picture) in the IP header section, while UDP uses protocol number 17 (0x11).

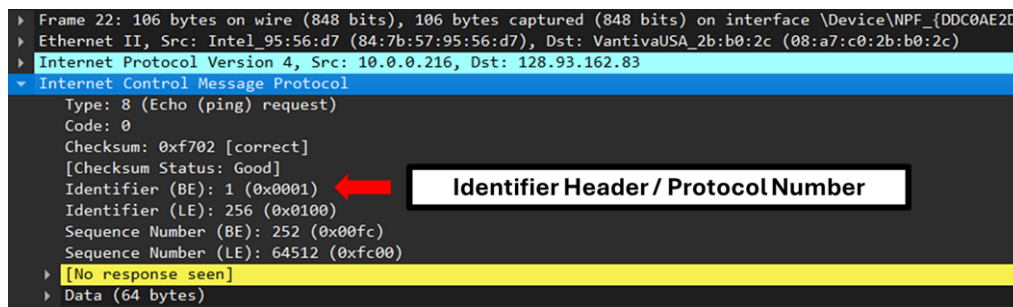


Figure 4: Protocol number as seen in the header of the ICMP packet

3) Examine the ICMP echo packet in your screenshot. Is this different from the ICMP ping query packets in the first half of this lab? If yes, how so?

The fundamental structure of an ICMP Echo packet remain the same, Type 8, Code 0. However, for traceroute, the main difference comes in the form of the Time-To-Live (TTL) value of the IP header. Each successive traceroute packet has an incrementing TTL (1, 2, 3, ...and so on.) The ping command typically sends packets with a high default TTL, which in my specific case was 128.

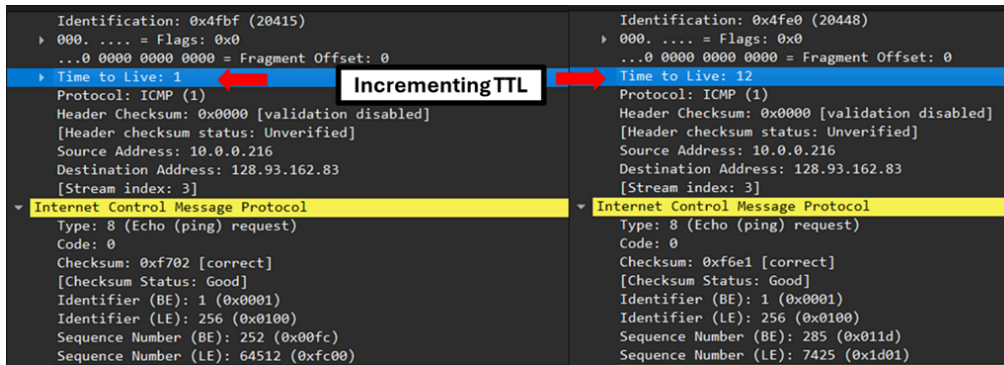


Figure 5: The first and the twelfth packet sent comparison. Time to live is increasing with each packet.

4) **Examine the ICMP error packet in your screenshot. It has more fields than the ICMP echo packet. What is included in those fields?**

When you look at the ICMP error packet in the screenshot, you'll notice it contains more information than a standard ICMP echo packet. After the main IP header, which is type 11 (TTL exceeded error) an additional subsection is present in the data. These extra fields are the full IP header from the original echo packet that caused the error. This info helps identify both the source and destination of that problematic packet.

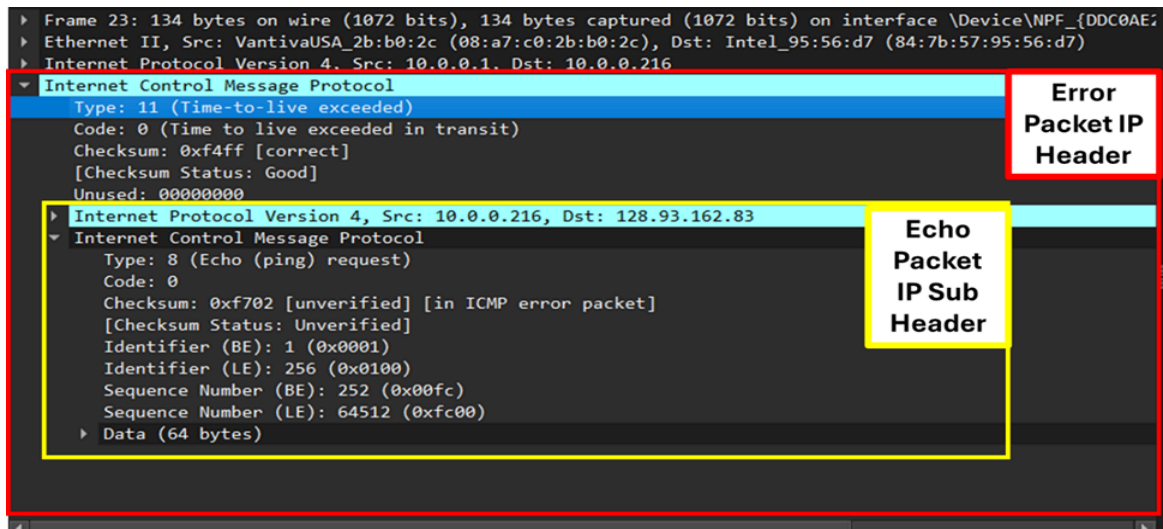


Figure 6: Error packet having a copy of the echo packet that caused the error.

5) **Examine the last three ICMP packets received by the source host. How are these packets different from the ICMP error packets? Why are they different?**

The last three ICMP packets received by the source host are ICMP Echo Reply packets (Type 0), which greatly differ from the ICMP error packets (Type 11). The Echo Replies indicate that the traceroute message was successful. They are a direct response from the final destination host, confirming receipt of a probe.

GitHub Repo: <https://github.com/BcAyush/Computer-Communications-Labs>

Team Members:

Team Name: Bit Busters

Ayush BC, Ernest Gusho, Nicolas Ramos

In contrast, Type 11 error packets are generated by intermediate routers when a probe's TTL expires, and they include a copy of the original packet's IP; something not needed in the Echo Reply.

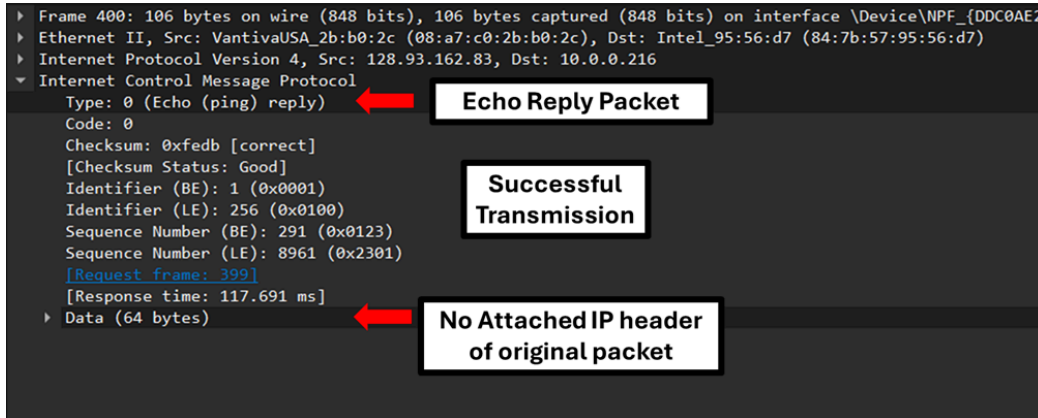


Figure 7: Successful echo reply packets

- 6) Within the traceroute measurements, is there a link whose delay is significantly longer than others? Refer to the screenshot in Figure 4, is there a link whose delay is significantly longer than others?

Yes, there is a link whose delay is significantly longer than others. It happens during the hop to link 9. This link (118 ms, 116 ms, 117 ms) and the proceeding ones have larger delays compared to the previous links (26 ms, 24 ms, 28 ms). This is likely due to the intercontinental network crossing. Based on the router hostnames, this transition appears to be between a U.S. based router in Georgia and one located in France.

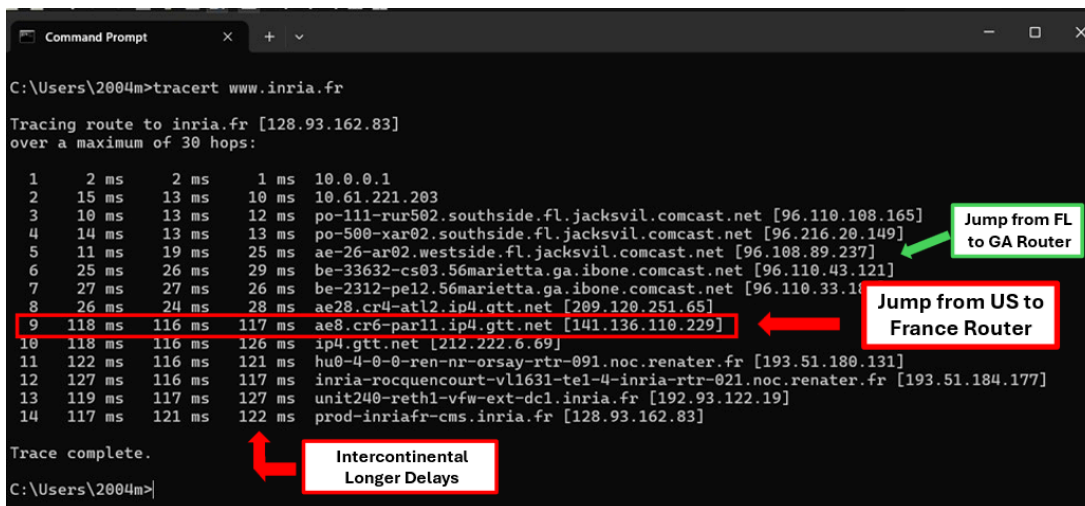


Figure 8: Command prompt window showing the hops. The router location can be easily derived based on the latency and domain names.

GitHub Repo: <https://github.com/BcAyush/Computer-Communications-Labs>

Team Members:

Team Name: Bit Busters

Ayush BC, Ernest Gusho, Nicolas Ramos

7) **On the basis of the router names, can you guess the location of the two routers on the end of this link?**

Yes, we can guess the location of the two routers based on their names. As seen in the picture above the link 8 router name contains “.atl2.”, making it based on Atlanta, Georgia, USA. The previous links can also be traced based on their names to go from Jacksonville, FL to Marietta, GA and then Atlanta, GA. Link 9, where the big delay happens, contains “ – par11” in its name, making the router based on Paris, France. The proceedings links also contain “.fr”, confirming France as the router location.

Part 3: ICMP and Traceroute using Pingplotter

1) What is the IP address of your host? What is the IP address of the target destination host?

The IP address of my host is 10.0.0.22 and the IP address of the target destination host is 128.93.162.83 as seen in blue box in screenshot below:

```
▶ Frame 7: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface \Device\NPF_{01FC5156-26
▶ Ethernet II, Src: Intel_cf:c8:3a (64:49:7d:cf:c8:3a), Dst: VantivaUSA_40:11:9f (40:75:c3:40:11:9f)
▶ Internet Protocol Version 4, Src: 10.0.0.22, Dst: 128.93.162.83
▼ Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0x24e7 [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 4438 (0x1156)
  Sequence Number (LE): 22033 (0x5611)
▶ [No response seen]
▶ Data (28 bytes)
```

2) If ICMP sent UDP packets instead (as in Unix/Linux), would the IP protocol number still be 01 for the probe packets? If not, what would it be?

If ICMP sent UDP packets instead of ICMP packets, as it does in Unix/Linux traceroute implementations, the IP protocol number would **not** remain 1. ICMP messages always use IP protocol number 1, while UDP packets use protocol number 17. Therefore, if the probe packets were UDP, the IP header in the trace would show 17 as the protocol field instead of 1. In this experiment, since the trace was performed on a Windows machine using ICMP, the protocol number observed in the IP header of the packets is correctly shown as 1.

Protocol: ICMP (1)

3) Examine the ICMP echo packet in your screenshot. Is this different from the ICMP ping query packets in the first half of this lab? If yes, how so?

The ICMP Echo packet captured in this part of the lab is fundamentally the same as the ICMP ping query packets observed in Part 1. Both are ICMP Echo Requests, with **Type 8** and **Code 0**, sent from the host to the target destination to test connectivity. The structure of the packet is identical, including fields like the identifier, sequence number, and checksum. As shown in the screenshot below, the ICMP Echo packet has

GitHub Repo: <https://github.com/BcAyush/Computer-Communications-Labs>

Team Members:

Team Name: Bit Busters

Ayush BC, Ernest Gusho, Nicolas Ramos

Type 8 and Code 0, just like the packets in Part 1.

```
Internet Control Message Protocol
Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0x24e5 [correct]
[Checksum Status: Good]
Identifier (BE): 1 (0x0001)
Identifier (LE): 256 (0x0100)
Sequence Number (BE): 4440 (0x1158)
Sequence Number (LE): 22545 (0x5811)
```

There may be minor differences in specific field values, such as the identifier or sequence number, since a different program generated them. Additionally, the TTL value in the IP header may differ depending on how each tool sets it. Functionally, though, both packets serve the same purpose: requesting an ICMP Echo Reply from the destination.

4) Examine the ICMP error packet in your screenshot. It has more fields than the ICMP echo packet. What is included in those fields?

The ICMP error packet captured in the trace is a *Time-to-live exceeded* message, which occurs when a probe packet's TTL (Time To Live) reaches zero at an intermediate router before reaching the destination. This ICMP error packet contains more fields than a simple Echo Request or Reply. Specifically, the ICMP header includes a **Type** field with a value of **11**, indicating *Time Exceeded*, and a **Code** field of **0**, meaning *Time to live exceeded in transit*. It also includes a **Checksum**, which verifies the integrity of the message, and an unused field set to zero. All can be seen in screenshot below:

```
Internet Control Message Protocol
Type: 11 (Time-to-live exceeded)
Code: 0 (Time to live exceeded in transit)
Checksum: 0xb6c1 [correct]
[Checksum Status: Good]
Unused: 00000000
```

In addition to these fields, the ICMP error packet embeds the original IP packet that triggered the error. This embedded portion includes the entire IP header and the first 8 bytes of the original datagram. This information allows the sender to identify which of its packets caused the error. In the example from my trace, the embedded IP header shows the **Source IP** as **10.0.0.22** (my host), the **Destination IP** as **128.93.162.83** (the target host), and a **TTL** of **1**, which caused the packet to expire at the router. These additional fields make the ICMP error packet more informative and larger than a basic Echo Request or Reply.

GitHub Repo: <https://github.com/BcAyush/Computer-Communications-Labs>

Team Members:

Team Name: Bit Busters

Ayush BC, Ernest Gusho, Nicolas Ramos

```
▼ Internet Protocol Version 4, Src: 10.0.0.22, Dst: 128.93.162.83
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 56
    Identification: 0x3be8 (15336)
  ▶ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
  ▶ Time to Live: 1
    Protocol: ICMP (1)
    Header Checksum: 0x5117 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.22
    Destination Address: 128.93.162.83
    [Stream index: 0]
```

5) Examine the last three ICMP packets received by the source host. How are these packets different from the ICMP error packets? Why are they different?

2069	98.196195	193.51.180.131	10.0.0.22	ICMP	110 Time-to-live exceeded (Time to live exceeded in transit)
2070	98.213970	10.0.0.22	128.93.162.83	ICMP	70 Echo (ping) request id=0x0001, seq=4987/31507, ttl=16 (reply in 2072)
2071	98.244879	193.51.184.177	10.0.0.22	ICMP	70 Time-to-live exceeded (Time to live exceeded in transit)
2072	98.332169	128.93.162.83	10.0.0.22	ICMP	70 Echo (ping) reply id=0x0001, seq=4987/31507, ttl=49 (request in 2070)

(# 2070 is sent by me thus, we are looking at 2069, 2071, and 2072)

The last three ICMP packets received by my host include two **Time Exceeded** messages and one **Echo Reply**. The two Time Exceeded packets (Type 11, Code 0) are error messages sent by intermediate routers to indicate that the TTL of the probe packet expired before reaching the destination. These packets also include the original IP header and first 8 bytes of the datagram that caused the error.

```
▼ Internet Control Message Protocol
  Type: 11 (Time-to-live exceeded)
  Code: 0 (Time to live exceeded in transit)
  Checksum: 0xf4ee [correct]
  [Checksum Status: Good]
  Unused: 00
  Length: 17
  [Length of original datagram: 68]
  Unused: 0000
```

The final ICMP packet is an **Echo Reply** (Type 0, Code 0), which is a response from the destination host indicating that it successfully received the Echo Request. Unlike the error packets, the Echo Reply does not include an embedded original datagram because it is not reporting an error — it is simply completing the request-reply exchange. This difference arises because the first two packets are error notifications

GitHub Repo: <https://github.com/BcAyush/Computer-Communications-Labs>

Team Members:

Team Name: Bit Busters

Ayush BC, Ernest Gusho, Nicolas Ramos

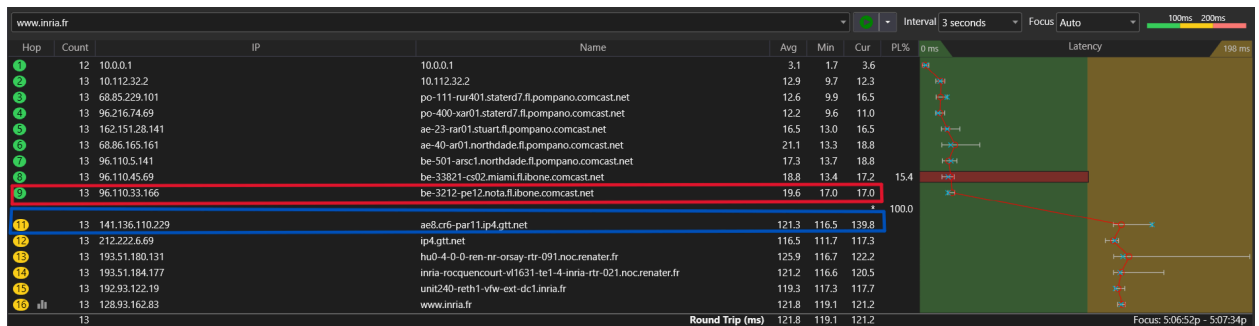
from routers along the path, while the last packet is the successful reply from the final destination.

```
Type: 0 (Echo (ping) reply)
Code: 0
Checksum: 0x2ac2 [correct]
[Checksum Status: Good]
Identifier (BE): 1 (0x0001)
Identifier (LE): 256 (0x0100)
Sequence Number (BE): 4987 (0x137b)
Sequence Number (LE): 31507 (0x7b13)
```

- 6) Within the tracer measurements, is there a link whose delay is significantly longer than others? Refer to the screenshot in Figure 4, is there a link whose delay is significantly longer than others?

Yes, the PingPlotter measurements show a link where the round-trip time (RTT) is significantly longer than the earlier hops. In the first part of the path (hops 1–9), the RTTs remain low, under ~20 ms, indicating traffic within the local and regional network. However, starting at hop 11, the RTT jumps sharply to over ~120 ms and stays elevated through to the destination. This is clearly visible in the PingPlotter screenshot below, where the RTT at hop 9 (highlighted in red) is still low, while at hop 11 (highlighted in blue) the RTT increases significantly. This jump likely corresponds to the transatlantic link between the last U.S. router

([be-3212-pe12.nota.fl.ibone.comcast.net](#)) and the first European router ([ae8.cr6-par11.ip4.gtt.net](#)), as indicated by their hostnames.



- 7) On the basis of the router names, can you guess the location of the two routers on the end of this link?

Based on the router names in the PingPlotter trace, the two routers at the ends of the long-delay link appear to be located in the United States and Europe. The last low-latency router before the RTT jump is at hop 9, named [be-3212-pe12.nota.fl.ibone.comcast.net](#), which suggests it is located in Florida (fl) in the U.S. The first high-latency router after the jump is at hop 11, named [ae8.cr6-par11.ip4.gtt.net](#), where par11 likely refers to Paris, France. This naming convention and the significant increase in round-trip time between these two routers are consistent with the

GitHub Repo: <https://github.com/BcAyush/Computer-Communications-Labs>

Team Members:

Team Name: Bit Busters

Ayush BC, Ernest Gusho, Nicolas Ramos

long-distance, transatlantic link between the U.S. and Europe.

www.inria.fr										Inte
Hop	Count	IP		Name		Avg	Min	Cur	PL%	
1	12	10.0.0.1		10.0.0.1		3.1	1.7	3.6		
2	13	10.112.32.2		10.112.32.2		12.9	9.7	12.3		
3	13	68.85.229.101		po-111-rur401.staterd7.fl.pompano.comcast.net		12.6	9.9	16.5		
4	13	96.216.74.69		po-400-xar01.staterd7.fl.pompano.comcast.net		12.2	9.6	11.0		
5	13	162.151.28.141		ae-23-rar01.stuart.fl.pompano.comcast.net		16.5	13.0	16.5		
6	13	68.86.165.161		ae-40-ar01.northdade.fl.pompano.comcast.net		21.1	13.3	18.8		
7	13	96.110.5.141		be-501-arsc1.northdade.fl.pompano.comcast.net		17.3	13.7	18.8		
8	13	96.110.45.69		be-33821-cs02.miami.fl.libone.comcast.net		18.8	13.4	17.2	15.4	
9	13	96.110.33.166		be-3212-pe12.nota.fl.libone.comcast.net		19.6	17.0	17.0		
								*		100.0
11	13	141.136.110.229		ae8.cr6-par11.jp4.gtt.net		121.3	116.5	139.8		
12	13	212.222.6.69		ip4.gtt.net		116.5	111.7	117.3		
13	13	193.51.180.131		hu0-4-0-0-ren-nr-orsay-rtr-091.noc.renater.fr		125.9	116.7	122.2		
14	13	193.51.184.177		inria-rocquencourt-v1631-te1-4-inria-rtr-021.noc.renater.fr		121.2	116.6	120.5		
15	13	192.93.122.19		unit240-reth1-vfw-ext-dc1.inria.fr		119.3	117.3	117.7		
16	13	128.93.162.83		www.inria.fr		121.8	119.1	121.2		